VISIONLINK

INTERNAL GUIDE / V1.0 / 2026

GRADUATION PROJECT · INTERNAL TEAM GUIDE

VisionLink.

A wearable industrial assistant on Raspberry Pi 4B — five pages, written so any of us can speak about the whole project without re-reading the code.

PLATFORM	TEAM	COURSE	REPO
Raspberry Pi 4B	Alaa · Ali · Alaa · Defne	EEE492 · Spring 2026	alaamjaish/visionlink

What VisionLink is — in 90 seconds

VisionLink is a wearable industrial assistant for factory workers. A Raspberry Pi 4B, a Pi Camera 3, an I²S microphone + amplifier, an 8 Ω speaker, six tactile push-buttons, and a 10 000 mAh power bank — clipped to a hard hat or worn on a chest strap. The total parts cost is under €120.

The worker drives the device with six buttons within reach. Three buttons handle **documentation** (open a shift session, snap a photo or short video, record a voice note). Three buttons handle **AI assistance** (talk to a cloud AI through the speaker, add the camera, or trigger a one-press emergency SOS). Everything captured streams to a Supabase Postgres database in ~200 ms, and a supervisor watches the activity in real time on a Next.js dashboard.

6	3	11	<2.5s	~€110
BUTTONS	AI PROVIDERS	LIVE PANELS	VOICE → AI	PROTOTYPE BOM

Why it exists

Industrial work in 2026 still depends heavily on memory, paper, and radios. A maintenance technician encounters a non-trivial problem perhaps once an hour — an unfamiliar part, a fault code, a leak that needs reporting. Each interaction follows the same pattern: stop, walk to a workstation, describe verbally, wait, walk back. Three failure modes recur — **knowledge that does not transfer** when senior workers retire, **emergencies that are slow and confused** because injured workers cannot reliably operate phones, and **reporting that is ad-hoc** and never reaches the analytics layer. VisionLink targets all three: it is wearable so hands stay on the work, voice-first because typing is incompatible with PPE-gloved hands, and always-uploading because the value of a captured shift is only realised when the supervisor and the analytics layer can see it.

> **One sentence we keep coming back to:** electrical correctness is necessary but not sufficient — the wearable lives or dies on whether a worker in welding gloves can reach a button, and whether the AI actually called the tool when it claimed to.

02 The Six Buttons

Each button is a falling-edge GPIO interrupt on the Pi. `scripts/gpio_bridge.py` debounces (20 ms LOW + 40 ms HIGH), classifies the gesture (single / double / hold), and POSTs `/api/button/<n>/<event>` to the FastAPI dashboard. The dashboard's six async handlers do everything from there.

B1 — GPIO 23 · Pin 16

Documentation Session

Toggles a shift session in Supabase. Every photo, video, and voice note captured while it is open is grouped under the same `session_id`.

`Single — open` `Single again — close`

B2 — GPIO 25 · Pin 22

Photo / Video

Single press snaps a 1280×720 JPEG. Double press records a 5-second MP4. Both upload to the Supabase storage bucket and appear on the dashboard.

`Single — photo` `Double — 5s video`

B3 — GPIO 22 · Pin 15

Voice Note

Press to start recording, press again to stop. PCM 16 kHz from the I²S mic is wrapped in a WAV header and uploaded to the session.

`Press → record` `Press → upload`

B4 — GPIO 5 · Pin 29

AI Voice Agent

Bidirectional voice session with Gemini Live (default) or OpenAI Realtime. AI hears, replies through the speaker, calls six live tools.

`Single — start` `Double — stop`

B5 — GPIO 6 · Pin 31

AI Voice + Vision

Same as B4 but with the camera attached. Default mode is snap-on-press: each B5 single press injects a fresh JPEG into the AI conversation.

`Single — start / snap` `Double — stop`

B6 · SOS — GPIO 13 · Pin 33

Panic Mode

Double-press arms full SOS. Single press is a pocket-dial warning toast. Emails the safety officer, opens a calm AI session, auto-snaps a frame every 10 s.

`Single — warning` `Double — armed`

SOS flow — the most complex thing in the codebase

When the SOS is armed: **(1)** a row is inserted into `sos_events`; **(2)** a fire-and-forget task emails the safety officer via Gmail SMTP using the internal `send_report` tool with role `"safety officer"`; **(3)** any open B1 session is auto-closed; **(4)** an emergency AI session opens (Gemini by default, OpenAI `gpt-realtime-2` fallback with the warm `"marin"` voice); **(5)** the AI opens the call first — *"Alaa, I'm here with you. Help is on the way. Are you hurt? Where are you?"*; **(6)** camera frames auto-snap every `sos_photo_interval_s` (default 10 s, 4 s on OpenAI), upload to `sos/<id>/frame_<ts>.jpg`, and inject into the AI session; **(7)** the live transcript streams into `sos_events.live_transcript`; **(8)** the supervisor's `SosPanel` turns red and pulses; **(9)** a red SHUT OFF button flips `resolved=true` — the wearable polls every 2 s and tears down within ~2 s; **(10)** a hard timeout auto-resolves after `sos_max_duration_s` (default 600 s).

> **What we'll never cut:** voice agent fires tools reliably ✓; six buttons all do something useful ✓; email actually arrives in the supervisor's inbox during demo ✓; ops dashboard updates in real time on a second screen ✓.

03 The Hardware

Off-the-shelf consumer parts. A Raspberry Pi 4B does all the real-time control locally; the cloud handles persistence and the AI. The wearable plus its 10 000 mAh USB-C power bank weighs under 300 g and fits on a hard-hat strap.

Components

Component	Model	Role	Connection
Computer	Raspberry Pi 4B (8 GB)	Brain. All real-time control.	USB-C 5 V / 3 A
Camera	Pi Camera Module 3	Photos, videos, AI vision frames	CSI ribbon
Microphone	SPH0645LM4H I²S MEMS	Voice input	I²S — GPIO 18/19/20
Amplifier	MAX98357A I²S Class-D	Speaker driver	I²S — GPIO 18/19/21
Speaker	8 Ω 1 W oval (Adafruit-style)	Audio output	Screw terminals on amp
Buttons	6 × 12 mm tactile push	Worker input	GPIO 5, 6, 13, 22, 23, 25
Power	10 000 mAh USB-C PD bank	Mobile power	USB-C
Mount	3D-printed PLA + helmet bracket	Wearable form	M3 hardware

GPIO pin map (BCM)

Function	BCM	Pin	Notes
B1 — Doc Session	23	16	Falling edge, internal pull-up
B2 — Photo / Video	25	22	Falling edge, internal pull-up
B3 — Voice Note	22	15	Falling edge, internal pull-up
B4 — AI Voice	5	29	Falling edge, internal pull-up
B5 — AI Voice + Vision	6	31	Falling edge, internal pull-up
B6 — SOS	13	33	Falling edge, internal pull-up
Mic + Amp BCLK (shared)	18	12	I²S bit clock
Mic + Amp LRCLK (shared)	19	35	I²S word-select clock
Mic DOUT	20	38	Mic audio bits
Amp DIN	21	40	Amp audio bits

Wiring philosophy

Each button is wired between its GPIO pin and ground. Pressing shorts the pin to ground; the Pi's internal pull-up detects a falling edge. The microphone and amplifier share the same I²S clock lines (BCLK + LRCLK) but have separate data lines — so all audio runs on four wires plus power and ground. Total wiring fits comfortably on a single breadboard during prototyping.

Cost — TRY (May 2026, 1 USD ≈ 45.4 TRY)

Line	Prototype (TRY)	At 1 K MOQ (TRY)
SBC (Pi 4B / CM4 Lite)	4 985	2 000
Camera	1 665	600
Mic + amp + speaker	1 114	250
Power (bank / cells)	2 000	600
Buttons + wiring + storage	578	200
Enclosure + mount + cooling	670	210
Assembly + test + certification	—	330
Shipping / KDV slack	555	—
Total	**≈ 11 657 (~$257)**	**≈ 4 190 (~$92)**

Recurring per-worker cloud cost: ≈ 9 500 TRY/month (~$209), dominated by OpenAI Realtime audio (~$165 of that). Cutting active voice to 30 min/day drops it to ~$95/month. The realtime audio API decides whether per-seat pricing works.

04 Software, AI, and the Dashboards

Three processes on the Pi

Independent processes give isolation: a crash in one cannot kill the others. `audio_worker` is a subprocess of the FastAPI dashboard, so a C-level ALSA assertion crashes only the worker — the parent restarts it within ~1.5 s.

The stack

- **FastAPI + Uvicorn** — async Python HTTP/WebSocket server (dashboard/server.py, ~1 800 lines)
- **Next.js 16 + React 19 + Tailwind 4** — supervisor ops dashboard (ops/, port 3000)
- **Supabase** — managed Postgres + Storage + Realtime, used by both Python (supabase-py) and the browser (@supabase/supabase-js)
- **picamera2** — modern Pi Camera library with hardware MMAL acceleration
- **RPi.GPIO + lgpio + gpiozero** — falling-edge GPIO with debounce
- **sounddevice + custom AudioBridge** — low-latency I²S audio path, half-duplex mic muting during TTS (no AEC)
- **google-genai SDK + OpenAI SDK** — bidirectional streaming voice + vision
- **smtplib (Gmail) / resend** — email delivery, Gmail by default

AI providers (currently configured)

- **Gemini Live** — `gemini-3.1-flash-live-preview` for streaming voice + vision. Default for B4, B5, SOS.
- **Gemini text/vision fallback** — `gemini-2.5-flash`
- **OpenAI Realtime** — `gpt-realtime-2`. Voice `"marin"` for the SOS path, `reasoning_effort="medium"`, output speed 1.2×.
- **Site chatbot** — `gemini-3.5-flash` (released 2026-05-19 at Google I/O 2026, $1.50/$9 per 1 M tokens, 1 M-token context)
- **Anthropic Claude (planned)** — wired but disabled; targeted for email drafting

The six AI tools (shared by Gemini and OpenAI sessions)

#	Tool	What it does
1	`lookup_component(query)`	Search components catalog by part_code (exact) then name (fuzzy). Returns ≤3 rows.
2	`log_incident(description, category?, severity?, location?)`	Insert into `incidents`. Categories: safety/equipment/leak/damage/other. Severities: low/medium/high/critical.
3	`mark_task_complete(task_query)`	Fuzzy-match pending tasks. Returns `ambiguous=true` with options when multiple match.
4	`get_my_assignments(include_complete?)`	List worker tasks, ordered by priority then due date.
5	`request_part(part_query, quantity?, urgency?, reason?)`	Submit to `part_requests`. Auto-matches against components.
6	`send_report(report_name, recipient_role?, recipient_name?, recipient_email?, custom_message?)`	Find template + recipient → inject live data → Gmail SMTP send → audit-log to `sent_reports`.

> **Three absolute rules** in the system prompt: (1) **Never lie about completing actions** — completion phrases require an actual tool call; (2) **Verb triggers** — *log, mark, send, request* fire the matching tool immediately; (3) **One clarifying question max** — commit with reasonable defaults instead of stalling.

The 11 Ops Dashboard panels

/ (Command Center) — 9 panels:

- `SosPanel` — live alarm + shutdown
- `WearableSettingsPanel` — remote config
- `IncidentsPanel` — safety/equipment logs
- `TasksPanel` — worker tasks
- `PartsPanel` — part requests
- `ComponentsPanel` — parts catalog
- `ManagersPanel` — supervisor directory
- `ReportTemplatesPanel` — email templates
- `SentReportsPanel` — email audit trail

/documentation — 2 panels:

- `SessionsPanel` — shifts, durations, per-kind counts
- `CapturesPanel` — tabbed grid (All / Photos / Videos / Voice notes / Orphans)

Database tables: `components`, `worker_tasks`, `incidents`, `part_requests`, `managers`, `report_templates`, `sent_reports`, `sessions`, `session_assets`, `sos_events`, `wearable_settings`.

05 The Demo, the Team, and the Numbers

Demo storyline

One worker, one supervisor, one wearable, one screen. The demo runs ~3 minutes and is built to land four distinct moments.

Moment 1 — Lookup that works. Worker presses **B4**. *"What's the torque spec on the gearbox M8 bolts?"* Wearable replies in ~1.2 s with the exact value from the components table. The tool-fired badge lights up on the voice-command-center screen.

Moment 2 — The AI sees. Worker presses **B5** and points at a QR-tagged part. *"What is this and when was it last maintained?"* The AI describes the part from the live camera frame and reads its maintenance record.

Moment 3 — Email arrives. Worker presses **B4** again. *"Send an email to the supervisor saying we completed the pump A3 inspection today, found two minor leaks on valve B7, and need replacement gaskets by Friday."* The wearable says *"One sec, drafting that now…"* — pause ~4 s — *"Email sent to Mr. Acharya."* The supervisor's inbox pings live on the second screen.

Moment 4 — SOS. Worker double-clicks **B6**. The supervisor's `SosPanel` turns red and pulses. A live AI conversation begins: *"Alaa, I'm here with you. Help is on the way."* Frames auto-snap every 10 s. The supervisor hits SHUT OFF. The wearable falls silent in ~2 s.

Performance numbers we can quote

Path	Mean	How measured
Button press → handler	35 ms	20 trials
AI request → first audio token (Gemini Live)	0.9 s	20 trials
AI request → first audio token (OpenAI Realtime)	0.7 s	20 trials
Single tool round-trip	180 ms	per-tool integration test
Photo → Supabase URL ready	1.4 s	stopwatch
SOS trigger → email in inbox	4.2 s	10 trials
Supervisor shutoff → wearable teardown	1.8 s	10 trials
6-hour soak — misclassification rate	0.93 %	1 184 events
6-hour soak — clean AI session close	100 % (38 / 38)	synthetic load

Team

SOFTWARE

Alaa Abo Jeesh

Software architecture, FastAPI backend, AI integration (Gemini Live + OpenAI Realtime), the tool-calling layer, and the Next.js ops dashboard. System prompts and soak-test design.

SOFTWARE

Ali Salih Yıldırım

Software development. Co-built the wearable's Python runtime and the supervisor dashboard. Integration testing of the voice and tool-calling paths.

HARDWARE + TESTING

Alaa Ali

Hardware integration (Pi assembly, I^2S wiring, button matrix), 3D-printed enclosure and helmet bracket, audio path bring-up, and end-to-end acceptance testing of every button.

HARDWARE + TESTING

Fatma Defne Dolaz

Hardware integration and helmet-mount fit-up, soak-test execution, end-to-end acceptance testing, and demo-day preparation. Requirements traceability across EEE491 → EEE492.

Open questions to lock in before demo

- Which exact teacher email goes into the `managers` row for "supervisor" and "safety officer"?
- Which QR-tagged components are printed on the demo poster (which `part_codes`)?
- Demo audio routing — are we using the wearable's speaker or a Bluetooth speaker for jury audibility?

- Battery — does the 10 000 mAh bank survive the full 30-minute jury slot under sustained AI streaming?
- Network — is the demo room on a Wi-Fi the Pi has been onboarded to, or do we need to bring our own hotspot?